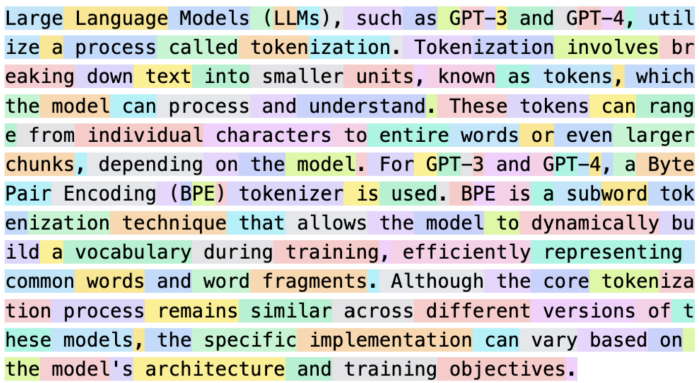
Deep Learning for NLP II

Tokenization



Learning goals

- Motivation for subword tokenization
- Overview of BPE and Unigram
- Open questions in tokenization

OUTLINE

Subword Tokenization Basics

Overview of BPE

Overview of Unigram

Current Issues in Tokenization Research

OUTLINE

Subword Tokenization Basics

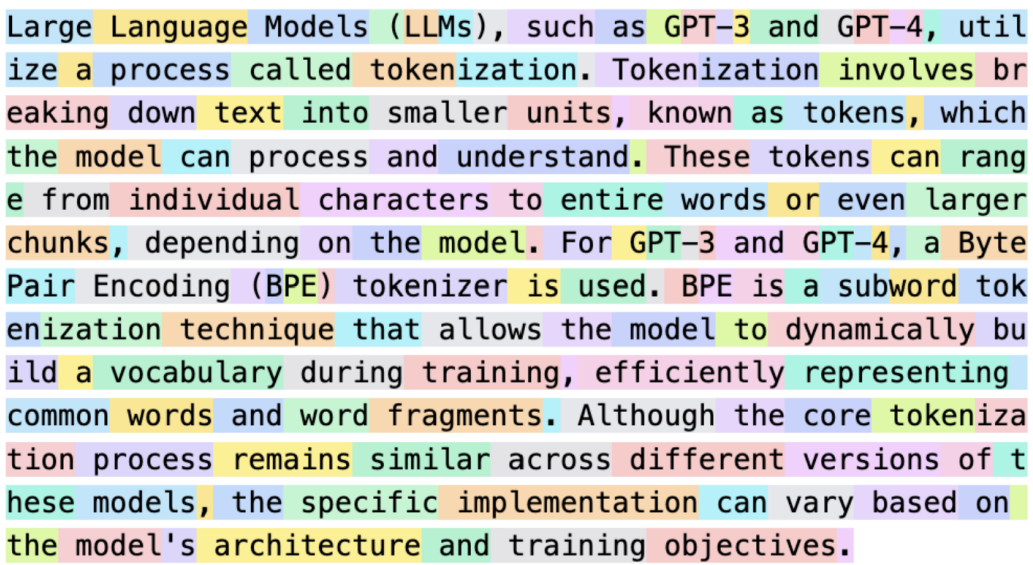
Overview of BPE

Overview of Unigram

Current Issues in Tokenization Research

MOTIVATION

- An LLM cannot simply digest the input text as is (infinite number of possible input texts)
- We need to map the input text to a sequence of units that the LLM “knows” (i.e., that correspond to fixed input representations)
- This step is commonly referred to as **tokenization**, and the units known to the LLM as **tokens**



TRY IT YOURSELF!

- Experiment with different types of input (e.g., different languages)
- What do you observe?
- How would you characterize the generated tokens?



`https://platform.openai.com/tokenizer`

TYPES OF TOKENIZATION

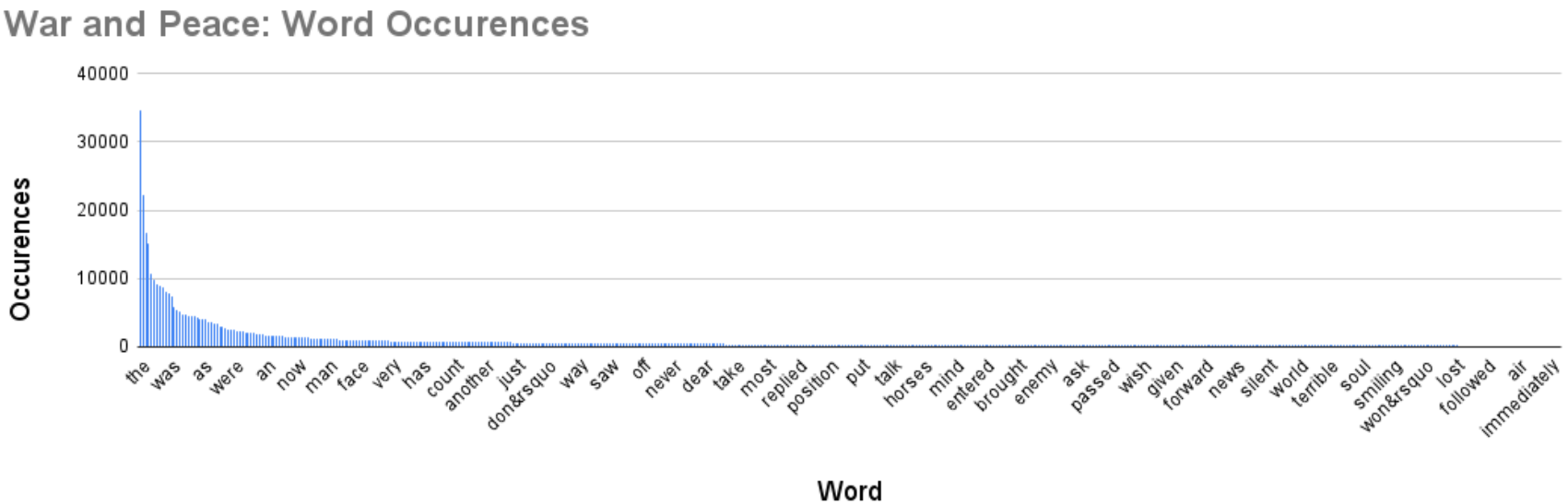
- **Word-level** tokenization
- **Character-level** (or **byte-level**) tokenization
- **Subword-level** tokenization $\leftarrow$ used by most LLMs

WORD-LEVEL TOKENIZATION

- Idea: split the input text into a sequence of **words**
- Widely used in pre-LLM NLP (e.g., static word embeddings)
- Pros:
  - Relatively simple to implement
  - Exact lexical representations
- Cons:
  - Large LLM vocabularies are computationally very expensive
  - **Heaps' law**: full coverage of all words impossible $\rightarrow$ many out-of-vocabulary (OOV) words

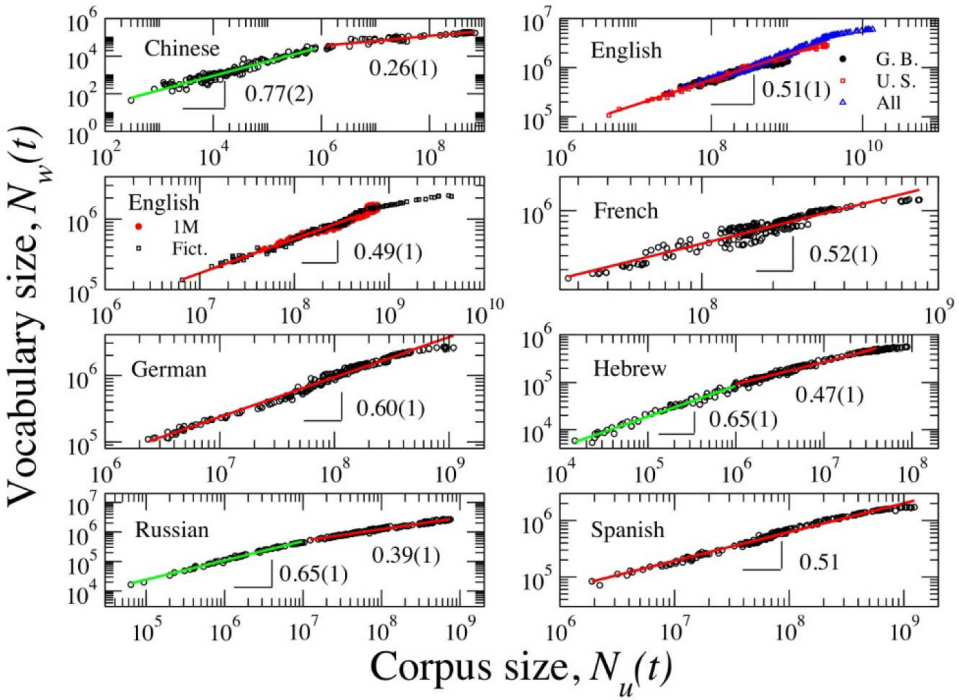
ZIPF'S LAW RECAP

- Mathematical form (simplified): $p(w_n) \propto 1/n$
  - p: relative frequency of a word
  - w_n: word at frequency rank n in a text
- Intuition: the second most common word occurs about half as often as the first, the third about a third as often, etc.



HEAPS' LAW

- Mathematical form: $V(n) = Kn^{\beta}$
  - V: vocabulary size (i.e., number of distinct word types)
  - n: corpus size (i.e., number of word tokens)
  - K, β: parameters (typically $10 < K < 100$, $0.4 < \beta < 0.6$)
- Intuition: the more text you read, the fewer new words you encounter, but you never stop encountering them



CHARACTER-LEVEL TOKENIZATION

- Idea: split the input text into a sequence of **characters** (or **bytes**)
- Pros:
  - Simple to implement
  - Leads to small token vocabularies
- Cons:
  - Lexical representations become a function of characters
  - Difficult to train in practice

SUBWORD-LEVEL TOKENIZATION

- Idea: split the input text into a mixed sequence of **words**, **subwords**, and **characters** (or **bytes**)
- Pros:
  - Relatively small token vocabularies (30K to 100K tokens)
  - Exact lexical representations for frequent words
  - Less frequent words represented as a (typically short) sequence of subwords and characters
- Cons:
  - Most complex tokenization approach

COMMON SUBWORD-LEVEL TOKENIZERS

- **Byte-pair encoding (BPE)** ▸ Sennrich et al. (2016)[7]: GPT, Llama, Qwen
- **Unigram** ▸ Kudo (2018)[5]: ALBERT, XLNet, T5
- Two key questions:
  - Training: how to determine the vocabulary of tokens?
  - Inference: how to segment texts into tokens?

NOTATION

- $\mathbf{s} = \langle s_1, s_2, \ldots \rangle$: sequence of characters/bytes, such that $s_t \in \Sigma$ for a base alphabet Σ
- $\mathbf{v} = \langle v_1, v_2, \ldots \rangle$: sequence of **tokens**, such that $v_t \in \mathcal{V}$ for a finite set $\mathcal{V}$ called **vocabulary**
- $h : \Sigma^* \to \mathcal{V}^*$: **tokenization function**
- **s** can be reconstructed from **v** via a **detokenization function** $g : \mathcal{V}^* \to \Sigma^*$ (simple mapping like string concatenation)
- g is generally non-injective: there can be (and often are) multiple valid **v** for which g would lead to the same **s**
- $\mathcal{T}_\mathcal{V}(\mathbf{s}) = g^{-1}(\mathbf{s}) = \{\mathbf{v} \in \mathcal{V}^* : g(\mathbf{v}) = \mathbf{s}\}$: set of all valid token sequences that detokenize to **s**
- $h(\mathbf{s})$ is the **canonical tokenization**, whereas $\mathbf{v} \in \mathcal{T}_\mathcal{V}(\mathbf{s})$ with $\mathbf{v} \neq h(\mathbf{s})$ are **non-canonical tokenizations**

EXAMPLE

- $\mathcal{V} = \{\mathtt{h}, \mathtt{a}, \mathtt{t}, \mathtt{ha}, \mathtt{at}\}$
- $h(\textit{hat}) = \langle \mathtt{ha}, \mathtt{t} \rangle$
- $g(\langle \mathtt{ha}, \mathtt{t} \rangle) = \textit{hat}$
- $\mathcal{T}_{\mathcal{V}}(\textit{hat}) = \{\langle \mathtt{h}, \mathtt{a}, \mathtt{t} \rangle, \langle \mathtt{ha}, \mathtt{t} \rangle, \langle \mathtt{h}, \mathtt{at} \rangle\}$
- $\langle \mathtt{ha}, \mathtt{t} \rangle$ is the canonical tokenization
- $\langle \mathtt{h}, \mathtt{a}, \mathtt{t} \rangle$ is a non-canonical tokenization

OUTLINE

Subword Tokenization Basics

Overview of BPE

Overview of Unigram

Current Issues in Tokenization Research

BPE TRAINING

- Originally a **text compression algorithm**
- Initialization:
  - Base vocabulary: $\mathcal{V} = \Sigma$
  - Apply **normalization** and **pretokenization** (e.g., split on whitespace)
  - **Bottom-up** approach: Each **s** starts as its character sequence $\mathbf{v} = \langle s_1, \ldots, s_{|\mathbf{s}|} \rangle$
- Repeat until $|\mathcal{V}|$ reaches the target size:
  - Count adjacent token pairs $\langle u, w \rangle$ in the corpus
  - Take the most frequent pair (u^*, w^*), add the new token $u^* w^*$ to $\mathcal{V}$, and record the merge rule $(u^*, w^*) \rightarrow u^* w^*$
  - Replace every occurrence of $\langle u^*, w^* \rangle$ in the corpus by $u^* w^*$
- Output: vocabulary $\mathcal{V}$ and an ordered list of merge rules

BPE INFERENCE

- Given a string $\mathbf{s}$, start from its characters: $\mathbf{v} = \langle s_1, \ldots, s_{|\mathbf{s}|} \rangle$
- Apply the merge rules in the order they were learned: for each rule $(u, w) \rightarrow uw$, replace every adjacent pair $\langle u, w \rangle$ in $\mathbf{v}$ by uw
- The result is the tokenization $h(\mathbf{s})$, with $g(h(\mathbf{s})) = \mathbf{s}$
- BPE thus deterministically picks one element of $\mathcal{T}_{\mathcal{V}}(\mathbf{s})$

Training Data

```
Proof of the Milky Way
consisting of many stars
came in 1610 when Galileo
Galilei used a telescope to
study the Milky Way and
discovered that it is
composed of a huge number
of faint stars.
```

Given training data D

Training Data

```
{Proof, _of, _the, _Milky,
_Way, _consisting, _of,
_many, _stars, _came, _in,
_1610, _when, _Galileo,
_Galilei, _used, _a,
_telescope, _to, _study,
_the, _Milky, _Way, _and,
_discovered, _that, _it,
_is, _composed, _of, _a,
_huge, _number, _of,
_faint, _stars.}
```

Pretokenize D by splitting on whitespace

Training Data

_ P r o o f, _ o f, _ t h
e, _ M i l k y, _ W a y, _
c o n s i s t i n g, _ o f,
_ m a n y, _ s t a r s, _ c
a m e, _ i n, _ 1 6 1 0, _
w h e n, _ G a l i l e o, _
G a l i l e i, _ u s e d, _
a, _ t e l e s c o p e, _ t
o, _ s t u d y, _ t h e, _
M i l k y, _ W a y, _ a n
d, _ d i s c o v e r e d, _
t h a t, _ i t, _ i s, _ c
o m p o s e d, _ o f, _ a,
_ h u g e, _ n u m b e r, _
o f, _ f a i n t, _ s t a r
s .

Split D into sequence of bytes

Training Data

_ P r o o f, _ o f, _ t h e, _ M i l k y, _ W a y, _ c o n s i s t i n g, _ o f, _ m a n y, _ s t a r s, _ c a m e, _ i n, _ 1 6 1 0, _ w h e n, _ G a l i l e o, _ G a l i l e i, _ u s e d, _ a, _ t e l e s c o p e, _ t o, _ s t u d y, _ t h e, _ M i l k y, _ W a y, _ a n d, _ d i s c o v e r e d, _ t h a t, _ i t, _ i s, _ c o m p o s e d, _ o f, _ a, _ h u g e, _ n u m b e r, _ o f, _ f a i n t, _ s t a r s .

Pair counts

_ t	12335282
t h	10067390
_ a	9319062
h e	8771183
i n	8024060
e r	6517430
a n	6315205
r e	6031043
o n	5261131
_ i	5209828

Vocabulary

Training Data

_ P r o o f, _ o f, _ t h e, _ M i l k y, _ W a y, _ c o n s i s t i n g, _ o f, _ m a n y, _ s t a r s, _ c a m e, _ i n, _ 1 6 1 0, _ w h e n, _ G a l i l e o, _ G a l i l e i, _ u s e d, _ a, _ t e l e s c o p e, _ t o, _ s t u d y, _ t h e, _ M i l k y, _ W a y, _ a n d, _ d i s c o v e r e d, _ t h a t, _ i t, _ i s, _ c o m p o s e d, _ o f, _ a, _ h u g e, _ n u m b e r, _ o f, _ f a i n t, _ s t a r s .

Pair counts

_ t	12335282
t h	10067390
_ a	9319062
h e	8771183
i n	8024060
e r	6517430
a n	6315205
r e	6031043
o n	5261131
_ i	5209828

Vocabulary

_t

Training Data

_ P r o o f, _ o f, _t h e,
_ M i l k y, _ W a y, _ c o
n s i s t i n g, _ o f, _ m
a n y, _ s t a r s, _ c a m
e, _ i n, _ 1 6 1 0, _ w h
e n, _ G a l i l e o, _ G a
l i l e i, _ u s e d, _ a,
_t e l e s c o p e, _t o, _
s t u d y, _t h e, _ M i l
k y, _ W a y, _ a n d, _ d
i s c o v e r e d, _t h a
t, _ i t, _ i s, _ c o m p
o s e d, _ o f, _ a, _ h u
g e, _ n u m b e r, _ o f,
_ f a i n t, _ s t a r s .

Pair counts

_ t	12335282
t h	10067390
_ a	9319062
h e	8771183
i n	8024060
e r	6517430
a n	6315205
r e	6031043
o n	5261131
_ i	5209828

Vocabulary

_t

Training Data

_ P r o o f, _ o f, _t h e,
_ M i l k y, _ W a y, _ c o
n s i s t i n g, _ o f, _ m
a n y, _ s t a r s, _ c a m
e, _ i n, _ 1 6 1 0, _ w h
e n, _ G a l i l e o, _ G a
l i l e i, _ u s e d, _ a,
_t e l e s c o p e, _t o, _
s t u d y, _t h e, _ M i l
k y, _ W a y, _ a n d, _ d
i s c o v e r e d, _t h a
t, _ i t, _ i s, _ c o m p
o s e d, _ o f, _ a, _ h u
g e, _ n u m b e r, _ o f,
_ f a i n t, _ s t a r s .

Pair counts

_ a	9319062
h e	8771183
i n	8024060
e r	6517430
a n	6315205
r e	6031043
o n	5261131
_ i	5209828

Vocabulary

_t

Training Data

_ P r o o f, _ o f, _t h e,
_ M i l k y, _ W a y, _ c o
n s i s t i n g, _ o f, _ m
a n y, _ s t a r s, _ c a m
e, _ i n, _ 1 6 1 0, _ w h
e n, _ G a l i l e o, _ G a
l i l e i, _ u s e d, _ a,
_t e l e s c o p e, _t o, _
s t u d y, _t h e, _ M i l
k y, _ W a y, _ a n d, _ d
i s c o v e r e d, _t h a
t, _ i t, _ i s, _ c o m p
o s e d, _ o f, _ a, _ h u
g e, _ n u m b e r, _ o f,
_ f a i n t, _ s t a r s .

Pair counts

_ a	9319062
h e	8771183
i n	8024060
_t h	7897058
e r	6517430
a n	6315205
r e	6031043
o n	5261131
_ i	5209828

Vocabulary

_t

Training Data

_ P r o o f, _ o f, _t h e,
_ M i l k y, _ W a y, _ c o
n s i s t i n g, _ o f, _ m
a n y, _ s t a r s, _ c a m
e, _ i n, _ 1 6 1 0, _ w h
e n, _ G a l i l e o, _ G a
l i l e i, _ u s e d, _ a,
_t e l e s c o p e, _t o, _
s t u d y, _t h e, _ M i l
k y, _ W a y, _ a n d, _ d
i s c o v e r e d, _t h a
t, _ i t, _ i s, _ c o m p
o s e d, _ o f, _ a, _ h u
g e, _ n u m b e r, _ o f,
_ f a i n t, _ s t a r s .

Pair counts

_ a	9319062
h e	8771183
i n	8024060
_t h	7897058
e r	6517430
a n	6315205
r e	6031043
o n	5261131
_ i	5209828
_ o	5163783

Vocabulary

_t

Training Data

_ P r o o f, _ o f, _t h e,
_ M i l k y, _ W a y, _ c o
n s i s t i n g, _ o f, _ m
a n y, _ s t a r s, _ c a m
e, _ i n, _ 1 6 1 0, _ w h
e n, _ G a l i l e o, _ G a
l i l e i, _ u s e d, _ a,
_t e l e s c o p e, _t o, _
s t u d y, _t h e, _ M i l
k y, _ W a y, _ a n d, _ d
i s c o v e r e d, _t h a
t, _ i t, _ i s, _ c o m p
o s e d, _ o f, _ a, _ h u
g e, _ n u m b e r, _ o f,
_ f a i n t, _ s t a r s .

Pair counts

_ a	9319062
h e	8771183
i n	8024060
_t h	7897058
e r	6517430
a n	6315205
r e	6031043
o n	5261131
_ i	5209828
_ o	5163783

Vocabulary

_t

Training Data

_ P r o o f, _ o f, _t h e,
_ M i l k y, _ W a y, _ c o
n s i s t i n g, _ o f, _ m
a n y, _ s t a r s, _ c a m
e, _ i n, _ 1 6 1 0, _ w h
e n, _ G a l i l e o, _ G a
l i l e i, _ u s e d, _ a,
_t e l e s c o p e, _t o, _
s t u d y, _t h e, _ M i l
k y, _ W a y, _ a n d, _ d
i s c o v e r e d, _t h a
t, _ i t, _ i s, _ c o m p
o s e d, _ o f, _ a, _ h u
g e, _ n u m b e r, _ o f,
_ f a i n t, _ s t a r s .

Pair counts

_ a	9319062
h e	8771183
i n	8024060
_t h	7897058
e r	6517430
a n	6315205
r e	6031043
o n	5261131
_ i	5209828
_ o	5163783

Vocabulary

_t

_a

Training Data

_ P r o o f, _ o f, _t h e,
_ M i l k y, _ W a y, _ c o
n s i s t i n g, _ o f, _ m
a n y, _ s t a r s, _ c a m
e, _ i n, _ 1 6 1 0, _ w h
e n, _ G a l i l e o, _ G a
l i l e i, _ u s e d, _a,
_t e l e s c o p e, _t o, _
s t u d y, _t h e, _ M i l
k y, _ W a y, _a n d, _ d i
s c o v e r e d, _t h a t,
_ i t, _ i s, _ c o m p o s
e d, _ o f, _a, _ h u g e,
_ n u m b e r, _ o f, _ f a
i n t, _ s t a r s .

Pair counts

_ a	9319062
h e	8771183
i n	8024060
_t h	7897058
e r	6517430
a n	6315205
r e	6031043
o n	5261131
_ i	5209828
_ o	5163783

Vocabulary

_t

_a

Training Data

_ P r o o f, _ o f, _t h e,
_ M i l k y, _ W a y, _ c o
n s i s t i n g, _ o f, _ m
a n y, _ s t a r s, _ c a m
e, _ i n, _ 1 6 1 0, _ w h
e n, _ G a l i l e o, _ G a
l i l e i, _ u s e d, _a,
_t e l e s c o p e, _t o, _
s t u d y, _t h e, _ M i l
k y, _ W a y, _a n d, _ d i
s c o v e r e d, _t h a t,
_ i t, _ i s, _ c o m p o s
e d, _ o f, _a, _ h u g e,
_ n u m b e r, _ o f, _ f a
i n t, _ s t a r s .

Pair counts

h e	8771183
i n	8024060
_t h	7897058
e r	6517430
r e	6031043
o n	5261131
_ i	5209828
_ o	5163783

Vocabulary

_t
_a

Training Data

_ P r o o f, _ o f, _t h e,
_ M i l k y, _ W a y, _ c o
n s i s t i n g, _ o f, _ m
a n y, _ s t a r s, _ c a m
e, _ i n, _ 1 6 1 0, _ w h
e n, _ G a l i l e o, _ G a
l i l e i, _ u s e d, _a,
_t e l e s c o p e, _t o, _
s t u d y, _t h e, _ M i l
k y, _ W a y, _a n d, _ d i
s c o v e r e d, _t h a t,
_ i t, _ i s, _ c o m p o s
e d, _ o f, _a, _ h u g e,
_ n u m b e r, _ o f, _ f a
i n t, _ s t a r s .

Pair counts

h e	8771183
i n	8024060
_t h	7897058
e r	6517430
r e	6031043
o n	5261131
_ i	5209828
_ o	5163783
_ s	5035505
_ w	4523998

Vocabulary

_t
_a

Training Data

_ P r o o f, _ o f, _t h e,
_ M i l k y, _ W a y, _ c o
n s i s t i n g, _ o f, _ m
a n y, _ s t a r s, _ c a m
e, _ i n, _ 1 6 1 0, _ w h
e n, _ G a l i l e o, _ G a
l i l e i, _ u s e d, _a,
_t e l e s c o p e, _t o, _
s t u d y, _t h e, _ M i l
k y, _ W a y, _a n d, _ d i
s c o v e r e d, _t h a t,
_ i t, _ i s, _ c o m p o s
e d, _ o f, _a, _ h u g e,
_ n u m b e r, _ o f, _ f a
i n t, _ s t a r s .

Pair counts

h e	8771183
i n	8024060
_t h	7897058
e r	6517430
r e	6031043
o n	5261131
_ i	5209828
_ o	5163783
_ s	5035505
_ w	4523998

Vocabulary

_t

_a

Training Data

_ P r o o f, _ o f, _t h e,
_ M i l k y, _ W a y, _ c o
n s i s t i n g, _ o f, _ m
a n y, _ s t a r s, _ c a m
e, _ i n, _ 1 6 1 0, _ w h
e n, _ G a l i l e o, _ G a
l i l e i, _ u s e d, _a,
_t e l e s c o p e, _t o, _
s t u d y, _t h e, _ M i l
k y, _ W a y, _a n d, _ d i
s c o v e r e d, _t h a t,
_ i t, _ i s, _ c o m p o s
e d, _ o f, _a, _ h u g e,
_ n u m b e r, _ o f, _ f a
i n t, _ s t a r s .

Pair counts

h e	8771183
i n	8024060
_t h	7897058
e r	6517430
r e	6031043
o n	5261131
_ i	5209828
_ o	5163783
_ s	5035505
_ w	4523998

Vocabulary

_t
_a
he

Training Data

_ P r o o f, _ o f, _t he,
_ M i l k y, _ W a y, _ c o
n s i s t i n g, _ o f, _ m
a n y, _ s t a r s, _ c a m
e, _ i n, _ 1 6 1 0, _ w he
n, _ G a l i l e o, _ G a l
i l e i, _ u s e d, _a, _t
e l e s c o p e, _t o, _ s
t u d y, _t he, _ M i l k
y, _ W a y, _a n d, _ d i s
c o v e r e d, _t h a t, _
i t, _ i s, _ c o m p o s e
d, _ o f, _a, _ h u g e, _
n u m b e r, _ o f, _ f a i
n t, _ s t a r s .

Pair counts

h e	8771183
i n	8024060
_t h	7897058
e r	6517430
r e	6031043
o n	5261131
_ i	5209828
_ o	5163783
_ s	5035505
_ w	4523998

Vocabulary

_t
_a
he

Training Data

_ P r o o f, _ o f, _t he,
_ M i l k y, _ W a y, _ c o
n s i s t i n g, _ o f, _ m
a n y, _ s t a r s, _ c a m
e, _ i n, _ 1 6 1 0, _ w he
n, _ G a l i l e o, _ G a l
i l e i, _ u s e d, _a, _t
e l e s c o p e, _t o, _ s
t u d y, _t he, _ M i l k
y, _ W a y, _a n d, _ d i s
c o v e r e d, _t h a t, _
i t, _ i s, _ c o m p o s e
d, _ o f, _a, _ h u g e, _
n u m b e r, _ o f, _ f a i
n t, _ s t a r s .

Pair counts

i n	8024060
e r	6517430
r e	6031043
o n	5261131
_ i	5209828
_ o	5163783
_ s	5035505
_ w	4523998

Vocabulary

_t
_a
he

Training Data

_ P r o o f, _ o f, _t he, _ M i l k y, _ W a y, _ c o n s i s t i n g, _ o f, _ m a n y, _ s t a r s, _ c a m e, _ i n, _ 1 6 1 0, _ w he n, _ G a l i l e o, _ G a l i l e i, _ u s e d, _a, _t e l e s c o p e, _t o, _ s t u d y, _t he, _ M i l k y, _ W a y, _a n d, _ d i s c o v e r e d, _t h a t, _ i t, _ i s, _ c o m p o s e d, _ o f, _a, _ h u g e, _ n u m b e r, _ o f, _ f a i n t, _ s t a r s .

Pair counts

i n	8024060
r e	6031043
_t he	5605612
e r	5279258
o n	5261131
_ i	5209828
_ o	5163783
_ s	5035505
_ w	4523998

Vocabulary

_t
_a
he

Training Data

_ P r o o f, _ o f, _t he, _ M i l k y, _ W a y, _ c o n s i s t i n g, _ o f, _ m a n y, _ s t a r s, _ c a m e, _ i n, _ 1 6 1 0, _ w he n, _ G a l i l e o, _ G a l i l e i, _ u s e d, _a, _t e l e s c o p e, _t o, _ s t u d y, _t he, _ M i l k y, _ W a y, _a n d, _ d i s c o v e r e d, _t h a t, _ i t, _ i s, _ c o m p o s e d, _ o f, _a, _ h u g e, _ n u m b e r, _ o f, _ f a i n t, _ s t a r s .

Pair counts

i n	8024060
r e	6031043
_t he	5605612
e r	5279258
o n	5261131
_ i	5209828
_ o	5163783
_ s	5035505
_ w	4523998
a t	4424733

Vocabulary

_t

_a

he

Training Data

_ P r o o f, _ o f, _t he, _ M i l k y, _ W a y, _ c o n s i s t i n g, _ o f, _ m a n y, _ s t a r s, _ c a m e, _ i n, _ 1 6 1 0, _ w he n, _ G a l i l e o, _ G a l i l e i, _ u s e d, _a, _t e l e s c o p e, _t o, _ s t u d y, _t he, _ M i l k y, _ W a y, _a n d, _ d i s c o v e r e d, _t h a t, _ i t, _ i s, _ c o m p o s e d, _ o f, _a, _ h u g e, _ n u m b e r, _ o f, _ f a i n t, _ s t a r s .

Pair counts

i n	8024060
r e	6031043
_t he	5605612
e r	5279258
o n	5261131
_ i	5209828
_ o	5163783
_ s	5035505
_ w	4523998
a t	4424733

Vocabulary

_t

_a

he

Training Data

_ P r o o f, _ o f, _t he, _ M i l k y, _ W a y, _ c o n s i s t i n g, _ o f, _ m a n y, _ s t a r s, _ c a m e, _ i n, _ 1 6 1 0, _ w he n, _ G a l i l e o, _ G a l i l e i, _ u s e d, _a, _t e l e s c o p e, _t o, _ s t u d y, _t he, _ M i l k y, _ W a y, _a n d, _ d i s c o v e r e d, _t h a t, _ i t, _ i s, _ c o m p o s e d, _ o f, _a, _ h u g e, _ n u m b e r, _ o f, _ f a i n t, _ s t a r s .

Pair counts

i n	8024060
r e	6031043
_t he	5605612
e r	5279258
o n	5261131
_ i	5209828
_ o	5163783
_ s	5035505
_ w	4523998
a t	4424733

Vocabulary

_t
_a
he
in

Training Data

_ P r o o f, _ o f, _t he,
_ M i l k y, _ W a y, _ c o
n s i s t in g, _ o f, _ m
a n y, _ s t a r s, _ c a m
e, _ in, _ 1 6 1 0, _ w he
n, _ G a l i l e o, _ G a l
i l e i, _ u s e d, _a, _t
e l e s c o p e, _t o, _ s
t u d y, _t he, _ M i l k
y, _ W a y, _a n d, _ d i s
c o v e r e d, _t h a t, _
i t, _ i s, _ c o m p o s e
d, _ o f, _a, _ h u g e, _
n u m b e r, _ o f, _ f a
in t, _ s t a r s .

Pair counts

i n	8024060
r e	6031043
_t he	5605612
e r	5279258
o n	5261131
_ i	5209828
_ o	5163783
_ s	5035505
_ w	4523998
a t	4424733

Vocabulary

_t
_a
he
in

Training Data

_ P r o o f, _ o f, _t he, _ M i l k y, _ W a y, _ c o n s i s t in g, _ o f, _ m a n y, _ s t a r s, _ c a m e, _ in, _ 1 6 1 0, _ w he n, _ G a l i l e o, _ G a l i l e i, _ u s e d, _a, _t e l e s c o p e, _t o, _ s t u d y, _t he, _ M i l k y, _ W a y, _a n d, _ d i s c o v e r e d, _t h a t, _ i t, _ i s, _ c o m p o s e d, _ o f, _a, _ h u g e, _ n u m b e r, _ o f, _ f a in t, _ s t a r s .

Pair counts

r e	6031043
_t he	5605612
e r	5279258
o n	5261131
_ o	5163783
_ s	5035505
_ w	4523998
a t	4424733
o r	4162447
e s	4010515

Vocabulary

_t
_a
he
in

Training Data

_ P r o o f, _ o f, _t he,
_ M i l k y, _ W a y, _ c o
n s i s t in g, _ o f, _ m
a n y, _ s t a r s, _ c a m
e, _ in, _ 1 6 1 0, _ w he
n, _ G a l i l e o, _ G a l
i l e i, _ u s e d, _a, _t
e l e s c o p e, _t o, _ s
t u d y, _t he, _ M i l k
y, _ W a y, _a n d, _ d i s
c o v e r e d, _t h a t, _
i t, _ i s, _ c o m p o s e
d, _ o f, _a, _ h u g e, _
n u m b e r, _ o f, _ f a
in t, _ s t a r s .

Pair counts

r e	6031043
_t he	5605612
e r	5279258
o n	5261131
_ o	5163783
_ s	5035505
_ w	4523998
a t	4424733
o r	4162447
e s	4010515

Vocabulary

_t
_a
he
in

Training Data

_ P r o o f, _ o f, _t he,
_ M i l k y, _ W a y, _ c o
n s i s t in g, _ o f, _ m
a n y, _ s t a r s, _ c a m
e, _ in, _ 1 6 1 0, _ w he
n, _ G a l i l e o, _ G a l
i l e i, _ u s e d, _a, _t
e l e s c o p e, _t o, _ s
t u d y, _t he, _ M i l k
y, _ W a y, _a n d, _ d i s
c o v e r e d, _t h a t, _
i t, _ i s, _ c o m p o s e
d, _ o f, _a, _ h u g e, _
n u m b e r, _ o f, _ f a
in t, _ s t a r s .

Pair counts

r e	6031043
_t he	5605612
e r	5279258
o n	5261131
_ o	5163783
_ s	5035505
_ w	4523998
a t	4424733
o r	4162447
e s	4010515

Vocabulary

_t
_a
he
in
re

Training Data

_ P r o o f, _ o f, _t he,
_ M i l k y, _ W a y, _ c o
n s i s t in g, _ o f, _ m
a n y, _ s t a r s, _ c a m
e, _ in, _ 1 6 1 0, _ w he
n, _ G a l i l e o, _ G a l
i l e i, _ u s e d, _a, _t
e l e s c o p e, _t o, _ s
t u d y, _t he, _ M i l k
y, _ W a y, _a n d, _ d i s
c o v e re d, _t h a t, _ i
t, _ i s, _ c o m p o s e
d, _ o f, _a, _ h u g e, _
n u m b e r, _ o f, _ f a
in t, _ s t a r s .

Pair counts

r e	6031043
_t he	5605612
e r	5279258
o n	5261131
_ o	5163783
_ s	5035505
_ w	4523998
a t	4424733
o r	4162447
e s	4010515

Vocabulary

_t
_a
he
in
re

Training Data

_ P r o o f, _ o f, _t he, _ M i l k y, _ W a y, _ c o n s i s t in g, _ o f, _ m a n y, _ s t a r s, _ c a m e, _ in, _ 1 6 1 0, _ w he n, _ G a l i l e o, _ G a l i l e i, _ u s e d, _a, _t e l e s c o p e, _t o, _ s t u d y, _t he, _ M i l k y, _ W a y, _a n d, _ d i s c o v e re d, _t h a t, _ i t, _ i s, _ c o m p o s e d, _ o f, _a, _ h u g e, _ n u m b e r, _ o f, _ f a in t, _ s t a r s .

Pair counts

_t he	5605612
o n	5261131
_ o	5163783
_ s	5035505
e r	4754849
_ w	4523998
a t	4424733
o u	3838417
_ c	3831635
n d	3811435

Vocabulary

_t
_a
he
in
re

Training Data

_ P r o o f, _ o f, _t he, _ M i l k y, _ W a y, _ c o n s i s t in g, _ o f, _ m a n y, _ s t a r s, _ c a m e, _ in, _ 1 6 1 0, _ w he n, _ G a l i l e o, _ G a l i l e i, _ u s e d, _a, _t e l e s c o p e, _t o, _ s t u d y, _t he, _ M i l k y, _ W a y, _a n d, _ d i s c o v e re d, _t h a t, _ i t, _ i s, _ c o m p o s e d, _ o f, _a, _ h u g e, _ n u m b e r, _ o f, _ f a in t, _ s t a r s .

Pair counts

_t he	5605612
o n	5261131
_ o	5163783
_ s	5035505
e r	4754849
_ w	4523998
a t	4424733
o u	3838417
_ c	3831635
n d	3811435

Vocabulary

_t
_a
he
in
re

Training Data

_ P r o o f, _ o f, _t he, _ M i l k y, _ W a y, _ c o n s i s t in g, _ o f, _ m a n y, _ s t a r s, _ c a m e, _ in, _ 1 6 1 0, _ w he n, _ G a l i l e o, _ G a l i l e i, _ u s e d, _a, _t e l e s c o p e, _t o, _ s t u d y, _t he, _ M i l k y, _ W a y, _a n d, _ d i s c o v e re d, _t h a t, _ i t, _ i s, _ c o m p o s e d, _ o f, _a, _ h u g e, _ n u m b e r, _ o f, _ f a in t, _ s t a r s .

Pair counts

_t he	5605612
o n	5261131
_ o	5163783
_ s	5035505
e r	4754849
_ w	4523998
a t	4424733
o u	3838417
_ c	3831635
n d	3811435

Vocabulary

_t
_a
he
in
re
_the

Training Data

_ P r o o f, _ o f, _the, _ M i l k y, _ W a y, _ c o n s i s t in g, _ o f, _ m a n y, _ s t a r s, _ c a m e, _ in, _ 1 6 1 0, _ w he n, _ G a l i l e o, _ G a l i l e i, _ u s e d, _a, _t e l e s c o p e, _t o, _ s t u d y, _the, _ M i l k y, _ W a y, _a n d, _ d i s c o v e re d, _t h a t, _ i t, _ i s, _ c o m p o s e d, _ o f, _a, _ h u g e, _ n u m b e r, _ o f, _ f a in t, _ s t a r s .

Pair counts

_t he	5605612
o n	5261131
_ o	5163783
_ s	5035505
e r	4754849
_ w	4523998
a t	4424733
o u	3838417
_ c	3831635
n d	3811435

Vocabulary

_t
_a
he
in
re
_the

Training Data

_ P r o o f, _ o f, _the, _ M i l k y, _ W a y, _ c o n s i s t in g, _ o f, _ m a n y, _ s t a r s, _ c a m e, _ in, _ 1 6 1 0, _ w he n, _ G a l i l e o, _ G a l i l e i, _ u s e d, _a, _t e l e s c o p e, _t o, _ s t u d y, _the, _ M i l k y, _ W a y, _a n d, _ d i s c o v e re d, _t h a t, _ i t, _ i s, _ c o m p o s e d, _ o f, _a, _ h u g e, _ n u m b e r, _ o f, _ f a in t, _ s t a r s .

Pair counts

o n	5261131
_ o	5163783
_ s	5035505
e r	4754849
_ w	4523998
a t	4424733
o u	3838417
_ c	3831635
n d	3811435
o r	3661288

Vocabulary

_t
_a
he
in
re
_the

Training Data

_ P r o o f, _ o f, _the, _ M i l k y, _ W a y, _ c o n s i s t in g, _ o f, _ m a n y, _ s t a r s, _ c a m e, _ in, _ 1 6 1 0, _ w he n, _ G a l i l e o, _ G a l i l e i, _ u s e d, _a, _t e l e s c o p e, _t o, _ s t u d y, _the, _ M i l k y, _ W a y, _a n d, _ d i s c o v e re d, _t h a t, _ i t, _ i s, _ c o m p o s e d, _ o f, _a, _ h u g e, _ n u m b e r, _ o f, _ f a in t, _ s t a r s .

Pair counts

o n	5261131
_ o	5163783
_ s	5035505
e r	4754849
_ w	4523998
a t	4424733
o u	3838417
_ c	3831635
n d	3811435
o r	3661288

Vocabulary

_t

_a

he

in

re

_the

⋮

until we reach desired vocab size T

SHORTCOMINGS OF BPE

- **Greedy training**: merges are chosen one at a time and never undone, without any global objective
- **Frequency bias**: pairs of high-frequency bytes dominate the merges, since pair counts are not normalized
- **Leftover tokens**: intermediate tokens that were only stepping stones to longer ones stay in $\mathcal{V}$, even if they are rarely used
- **Deterministic inference**: exactly one tokenization $h(\mathbf{s})$ per string, with no way to sample other elements of $\mathcal{T}_{\mathcal{V}}(\mathbf{s})$

OUTLINE

Subword Tokenization Basics

Overview of BPE

Overview of Unigram

Current Issues in Tokenization Research

INTRODUCTION

- Tokenization treated as inference in a **latent variable model**
- Observed variable: **S**, a random variable taking values in the space of character/byte sequences **s** $\in \Sigma^*$
- Latent variable: **V**, a random variable taking values in the space of token sequences **v** $\in \mathcal{V}^*$
- Parameters: token probabilities $\phi \in \mathbb{R}_{\geq 0}^{|\mathcal{V}|}$ with $\sum_{v \in \mathcal{V}} \phi_v = 1$, where ϕ_v is the token probability for token v
- Training: Unigram learns both $\mathcal{V}$ and ϕ that (approximately) **maximize corpus log-likelihood**
- Inference: given **s**, Unigram chooses **v** that has the **highest posterior probability under the learned model**

GENERATIVE MODEL

- We assume token sequences $\mathbf{v} = \langle v_1, \ldots, v_m \rangle$ are generated as

$$v_t \overset{\text{i.i.d.}}{\sim} \text{Categorical}(\phi) \quad (t = 1, \ldots, m)$$

- We can then define the distribution of $\mathbf{V}$ as

$$P(\mathbf{V} = \mathbf{v}; \phi) \overset{\text{def}}{=} \prod_{t=1}^{|\mathbf{v}|} \phi_{v_t}$$

- Given the deterministic mapping g from tokens to strings, the distribution of $\mathbf{S}$ is a **pushforward** of the distribution of $\mathbf{V}$:

$$P(\mathbf{S} = \mathbf{s}; \phi) \overset{\text{def}}{=} \sum_{\mathbf{v} \in \mathcal{T}_{\mathcal{V}}(s)} P(\mathbf{V} = \mathbf{v}; \phi)$$

- NB: for the distribution of $\mathbf{V}$ to be a valid probability distribution on $\mathcal{V}^*$, we would have to specify a **length prior**, but this is rarely done in practice, so we will not consider it here

USEFUL RELATIONSHIPS BETWEEN V AND S

- Joint distribution of **V** and **S**:

$$P(\mathbf{V} = \mathbf{v}, \mathbf{S} = \mathbf{s}; \phi) = P(\mathbf{V} = \mathbf{v}; \phi)\ P(\mathbf{S} = \mathbf{s} \mid \mathbf{V} = \mathbf{v}; \phi)$$
$$= P(\mathbf{V} = \mathbf{v}; \phi)\ \mathbb{1}[g(\mathbf{v}) = \mathbf{s}]$$

- Marginalizing **V** out of the joint gives the distribution of **S**, identical to our pushforward definition:

$$P(\mathbf{S} = \mathbf{s}; \phi) = \sum_{\mathbf{v} \in \mathcal{V}^*} P(\mathbf{V} = \mathbf{v}; \phi)\ \mathbb{1}[g(\mathbf{v}) = \mathbf{s}]$$
$$= \sum_{\mathbf{v} \in \mathcal{T}_{\mathcal{V}}(\mathbf{s})} P(\mathbf{V} = \mathbf{v}; \phi)$$

- Applying Bayes' rule, we can easily derive the posterior probability over segmentations:

$$P(\mathbf{V} = \mathbf{v} \mid \mathbf{S} = \mathbf{s}; \phi) = \begin{cases} \dfrac{P(\mathbf{V} = \mathbf{v}; \phi)}{P(\mathbf{S} = \mathbf{s}; \phi)} & \text{if } \mathbf{v} \in \mathcal{T}_{\mathcal{V}}(\mathbf{s}) \\ 0 & \text{otherwise.} \end{cases}$$

EXAMPLE (CONTINUED)

- Continuing the example, suppose the token probabilities are

$$\phi_{\texttt{h}} = 0.3, \quad \phi_{\texttt{a}} = 0.1, \quad \phi_{\texttt{t}} = 0.25, \quad \phi_{\texttt{ha}} = 0.2, \quad \phi_{\texttt{at}} = 0.15$$

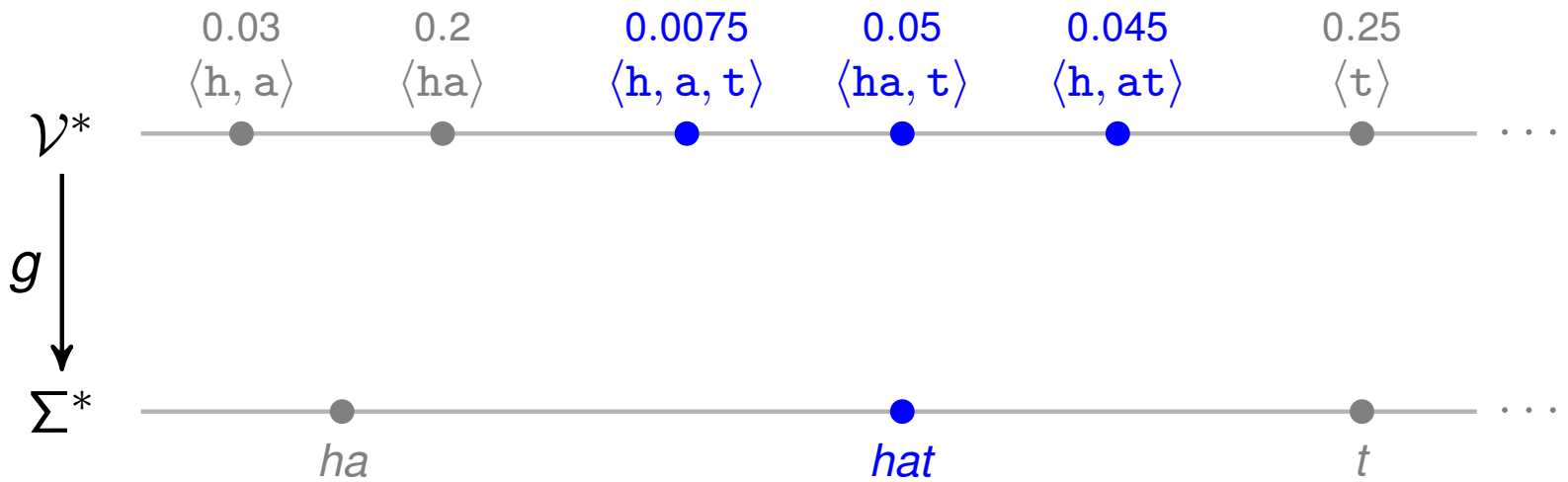
- Probability of each valid segmentation of *hat*:

$$\begin{aligned} P(\mathbf{V} = \langle \texttt{h}, \texttt{a}, \texttt{t} \rangle; \boldsymbol{\phi}) &= 0.3 \times 0.1 \times 0.25 = 0.0075 \\ P(\mathbf{V} = \langle \texttt{ha}, \texttt{t} \rangle; \boldsymbol{\phi}) &= 0.2 \times 0.25 = 0.05 \\ P(\mathbf{V} = \langle \texttt{h}, \texttt{at} \rangle; \boldsymbol{\phi}) &= 0.3 \times 0.15 = 0.045 \end{aligned}$$

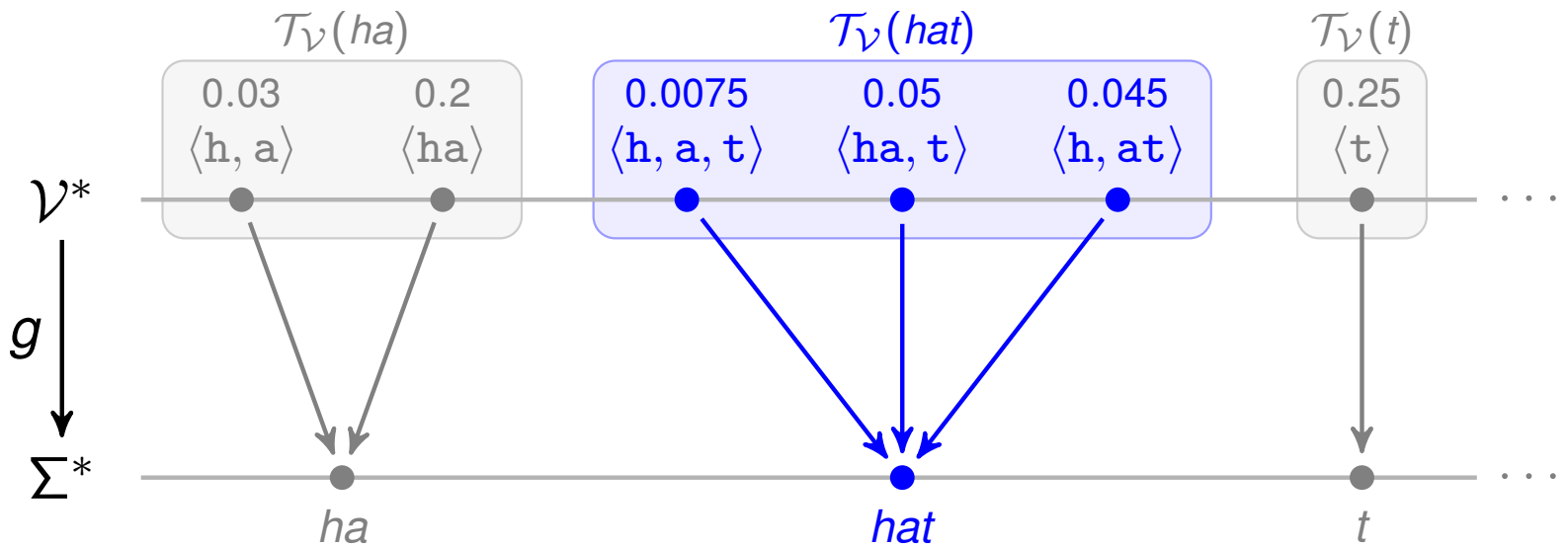
- Marginal probability of the string:

$$P(\mathbf{S} = \textit{hat}; \boldsymbol{\phi}) = 0.0075 + 0.05 + 0.045 = 0.1025$$

VISUALIZING THE PUSHFORWARD

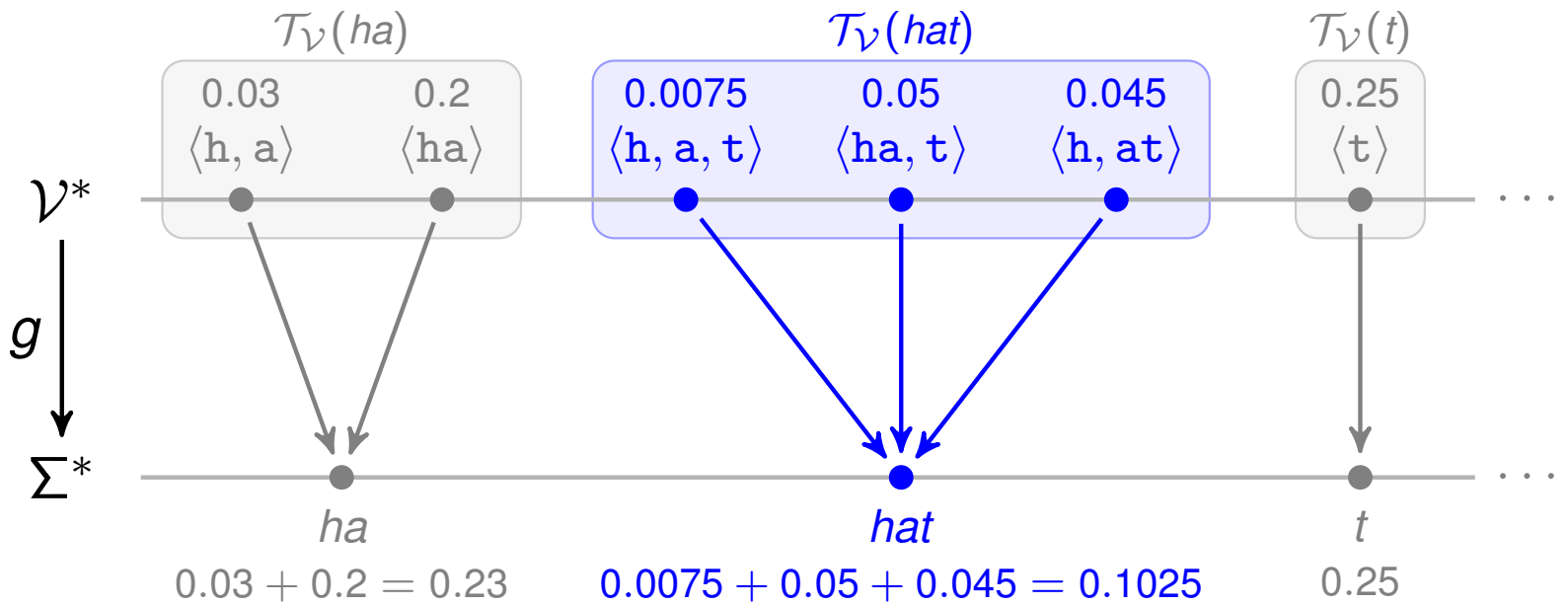


VISUALIZING THE PUSHFORWARD



- g maps each token sequence to exactly one string
- However, many token sequences can map to the same string **s**: its preimage $\mathcal{T}_\mathcal{V}(\mathbf{s}) = g^{-1}(\mathbf{s})$

VISUALIZING THE PUSHFORWARD



- g maps each token sequence to exactly one string
- However, many token sequences can map to the same string $\mathbf{s}$: its preimage $\mathcal{T}_\mathcal{V}(\mathbf{s}) = g^{-1}(\mathbf{s})$
- Each string collects the probability mass of its preimage:

$$P(\mathbf{S} = \mathbf{s}; \phi) = \sum_{\mathbf{v} \in \mathcal{T}_\mathcal{V}(\mathbf{s})} P(\mathbf{V} = \mathbf{v}; \phi)$$

EXAMPLE (CONTINUED)

- Dividing by $P(\mathbf{S} = \textit{hat}; \phi)$ gives the posterior:

$$P(\mathbf{V} = \langle \mathtt{h}, \mathtt{a}, \mathtt{t} \rangle \mid \mathbf{S} = \textit{hat}; \phi) = 0.0075/0.1025 \approx 0.073$$
$$P(\mathbf{V} = \langle \mathtt{ha}, \mathtt{t} \rangle \mid \mathbf{S} = \textit{hat}; \phi) = 0.05/0.1025 \approx 0.488$$
$$P(\mathbf{V} = \langle \mathtt{h}, \mathtt{at} \rangle \mid \mathbf{S} = \textit{hat}; \phi) = 0.045/0.1025 \approx 0.439$$

- $\langle \mathtt{h}, \mathtt{a}, \mathtt{t} \rangle$ gets little posterior mass, even though $\mathtt{h}$ and $\mathtt{t}$ are the two most probable tokens (a product of three probabilities is penalized relative to a product of two)
- The ranking of the two-token segmentations comes down to $\phi_{\mathtt{ha}}\phi_{\mathtt{t}}$ vs. $\phi_{\mathtt{h}}\phi_{\mathtt{at}}$: increasing $\phi_{\mathtt{at}}$ enough relative to $\phi_{\mathtt{ha}}$ would flip it
- The highest-posterior segmentation $\langle \mathtt{ha}, \mathtt{t} \rangle$ is the canonical tokenization $h(\textit{hat})$ from before

UNIGRAM INFERENCE

- For now, assume we know ϕ (or at least have estimates for it)
- At inference time, Unigram tokenizes **s** by choosing the segmentation with the **highest posterior probability**:

$$
\begin{aligned}
h_\phi(\mathbf{s}) &= \arg\max_{\mathbf{v} \in \mathcal{T}_\mathcal{V}(\mathbf{s})} P(\mathbf{V} = \mathbf{v} \mid \mathbf{S} = \mathbf{s}; \phi) \\
&= \arg\max_{\mathbf{v} \in \mathcal{T}_\mathcal{V}(\mathbf{s})} \frac{P(\mathbf{V} = \mathbf{v}; \phi)}{P(\mathbf{S} = \mathbf{s}; \phi)} \\
&= \arg\max_{\mathbf{v} \in \mathcal{T}_\mathcal{V}(\mathbf{s})} P(\mathbf{V} = \mathbf{v}; \phi) \\
&= \arg\max_{\mathbf{v} \in \mathcal{T}_\mathcal{V}(\mathbf{s})} \prod_{t=1}^{|\mathbf{v}|} \phi_{v_t}
\end{aligned}
$$

- Computed efficiently by a dynamic program (**Viterbi**)

EXAMPLE (CONTINUED)

- With ϕ from before, $\langle \texttt{ha}, \texttt{t} \rangle$ has the highest probability (0.05 vs. 0.045 and 0.0075), so $h_\phi(\textit{hat}) = \langle \texttt{ha}, \texttt{t} \rangle$
- Now swap the probabilities of $\texttt{t}$ and $\texttt{at}$: $\phi'_{\texttt{t}} = 0.15$, $\phi'_{\texttt{at}} = 0.25$, everything else unchanged

$$P(\mathbf{V} = \langle \texttt{h}, \texttt{a}, \texttt{t} \rangle; \phi') = 0.3 \times 0.1 \times 0.15 = 0.0045$$
$$P(\mathbf{V} = \langle \texttt{ha}, \texttt{t} \rangle; \phi') = 0.2 \times 0.15 = 0.03$$
$$P(\mathbf{V} = \langle \texttt{h}, \texttt{at} \rangle; \phi') = 0.3 \times 0.25 = 0.075$$

- The Viterbi segmentation flips: $h_{\phi'}(\textit{hat}) = \langle \texttt{h}, \texttt{at} \rangle$

UNIGRAM TRAINING BASICS

- Given a training corpus, we need to find $\mathcal{V}$ and ϕ
- **Top-down** approach:
  - We start with a large $\mathcal{V}$ (around 1M tokens, typically bytes/characters and all substrings up to some length)
  - ϕ initialized proportional to frequency in training corpus
  - Choice matters: if token is not in initial vocabulary, it will never be part of the tokenizer vocabulary!
  - We iteratively prune tokens until we reach the desired vocabulary size, while at the same time learning ϕ
- Core algorithm: **expectation-maximization (EM)**

EXPECTATION-MAXIMIZATION FOR UNIGRAM

- Training Unigram is a chicken-and-egg problem:
  - If we knew the segmentations, estimating ϕ would be easy
  - If we knew ϕ, we could infer the segmentations
- Expectation-maximization (EM) alternates between the two, starting from an initial guess $\phi^{(0)}$ (e.g., uniform):
  - **E-step**: use the current $\phi^{(n)}$ to compute the posterior over segmentations of each string ("soft" segmentations)
  - **M-step**: re-estimate $\phi^{(n+1)}$ from these soft segmentations (count tokens weighted by their posterior, then normalize)
- Repeat until ϕ converges

OBSERVED DATA LOG-LIKELIHOOD

- We want to find the parameters ϕ that maximize the log-likelihood of our data (maximum likelihood estimation)
- For a corpus $\mathcal{C} = \{\mathbf{s}_i\}_{i=1}^{K}$, we compute the log-likelihood of the **observed data** as

$$\begin{aligned}
\mathcal{L}(\mathcal{C}; \phi) &\stackrel{\text{def}}{=} \log \prod_{i=1}^{K} P(\mathbf{S} = \mathbf{s}_i; \phi) \\
&= \log \prod_{i=1}^{K} \sum_{\mathbf{v} \in \mathcal{T}_{\mathcal{V}}(\mathbf{s}_i)} P(\mathbf{V} = \mathbf{v}; \phi) \\
&= \sum_{i=1}^{K} \log \sum_{\mathbf{v} \in \mathcal{T}_{\mathcal{V}}(\mathbf{s}_i)} P(\mathbf{V} = \mathbf{v}; \phi)
\end{aligned}$$

- Optimizing $\mathcal{L}(\mathcal{C}; \phi)$ is difficult due to the log-sum structure

EXPECTED COMPLETE DATA LOG-LIKELIHOOD

- If we include the latent segmentations $\mathbf{v}_i$, we have the **complete data** $\mathcal{X} = \{(\mathbf{s}_i, \mathbf{v}_i)\}_{i=1}^{K}$, whose log-likelihood is

$$\mathcal{L}(\mathcal{X}; \phi) \stackrel{\text{def}}{=} \log \prod_{i=1}^{K} P(\mathbf{S} = \mathbf{s}_i, \mathbf{V} = \mathbf{v}_i; \phi)$$

$$= \sum_{i=1}^{K} \log P(\mathbf{S} = \mathbf{s}_i, \mathbf{V} = \mathbf{v}_i; \phi)$$

- We cannot compute $\mathcal{L}(\mathcal{X}; \phi)$ ($\mathbf{v}_i$ is unknown), but we *can* compute its expectation under the posterior with current parameters $\phi^{(n)}$:

$$Q(\phi; \phi^{(n)}) \stackrel{\text{def}}{=} \sum_{i=1}^{K} \mathbb{E}_{\mathbf{v} \sim P(\mathbf{V} | \mathbf{S} = \mathbf{s}_i; \phi^{(n)})} \big[\log P(\mathbf{S} = \mathbf{s}_i, \mathbf{V} = \mathbf{v}; \phi) \big]$$

$$= \sum_{i=1}^{K} \sum_{\mathbf{v} \in \mathcal{T}_{\mathcal{V}}(\mathbf{s}_i)} \underbrace{P(\mathbf{V} = \mathbf{v} \mid \mathbf{S} = \mathbf{s}_i; \phi^{(n)})}_{\text{weights from current } \phi^{(n)}} \underbrace{\log P(\mathbf{V} = \mathbf{v}; \phi)}_{\text{increase by adjusting } \phi}$$

QUICK RECAP

- We want to optimize the observed data log-likelihood:

$$\mathcal{L}(\mathcal{C}; \phi) = \sum_{i=1}^{K} \log \sum_{\mathbf{v} \in \mathcal{T}_{\mathcal{V}}(\mathbf{s}_i)} P(\mathbf{V} = \mathbf{v}; \phi)$$

- Intuition: make *corpus strings* more probable (where string probability is computed by marginalizing over valid segmentations)
- We actually optimize the expected complete data log-likelihood:

$$Q(\phi; \phi^{(n)}) = \sum_{i=1}^{K} \sum_{\mathbf{v} \in \mathcal{T}_{\mathcal{V}}(\mathbf{s}_i)} P(\mathbf{V} = \mathbf{v} \mid \mathbf{S} = \mathbf{s}_i; \phi^{(n)}) \log P(\mathbf{V} = \mathbf{v}; \phi)$$

- Intuition: make *valid segmentations* more probable, weighted by string frequency and current posterior

HOW Q RELATES TO $\mathcal{L}$

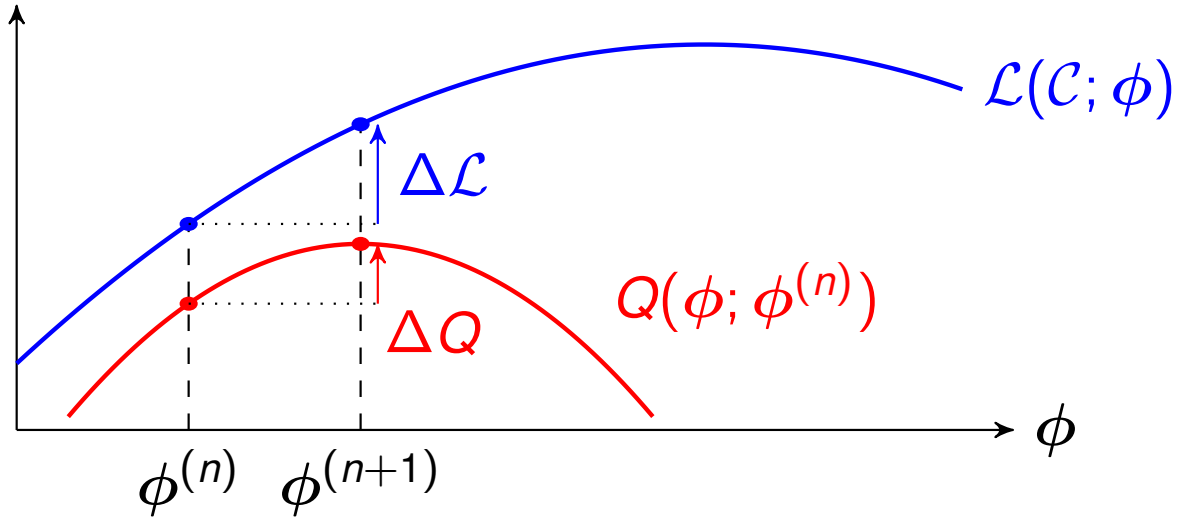
- It can be shown that $\mathcal{L}(\mathcal{C}; \phi)$ decomposes as follows:

$$\mathcal{L}(\mathcal{C}; \phi) = Q(\phi; \phi^{(n)}) + \underbrace{\sum_{i=1}^{K} H(\mathbf{V} \mid \mathbf{S} = \mathbf{s}_i; \phi^{(n)})}_{\text{entropy: constant w.r.t. } \phi}$$

$$+ \underbrace{\sum_{i=1}^{K} \mathrm{KL}\left(P(\mathbf{V} \mid \mathbf{S} = \mathbf{s}_i; \phi^{(n)}) \,\middle\|\, P(\mathbf{V} \mid \mathbf{S} = \mathbf{s}_i; \phi)\right)}_{\geq 0;\ 0 \text{ at } \phi = \phi^{(n)}}$$

- The KL term is 0 at $\phi^{(n)}$ and never negative, so pushing Q up from $\phi^{(n)}$ also pushes $\mathcal{L}(\mathcal{C}; \phi)$ up, by at least as much
- Q plus the entropy term is known as the **evidence lower bound**

INTUITION



- The vertical gap between $\mathcal{L}$ and Q at $\phi^{(n)}$ is the entropy term
- Maximizing Q moves us from $\phi^{(n)}$ to $\phi^{(n+1)}$
- $\mathcal{L}(\mathcal{C};\phi)$ rises by at least as much as Q: $\Delta\mathcal{L} \geq \Delta Q$
- The difference between $\Delta\mathcal{L}$ and ΔQ is the KL term at $\phi^{(n+1)}$, which is ≥ 0

E-STEP

- Goal: compute the expected complete data log-likelihood under our current parameters, $Q(\phi; \phi^{(n)})$
- Defining a count function $c_v(\mathbf{v}) \stackrel{\text{def}}{=} \sum_{t=1}^{|\mathbf{v}|} \mathbb{1}[v_t = v]$, we can write

$$\log P(\mathbf{V} = \mathbf{v}; \phi) = \sum_{t=1}^{|\mathbf{v}|} \log \phi_{v_t} = \sum_{v \in \mathcal{V}} c_v(\mathbf{v}) \log \phi_v$$

- Plugging this into Q gives

$$Q(\phi; \phi^{(n)}) = \sum_{v \in \mathcal{V}} \underbrace{\sum_{i=1}^{K} \sum_{\mathbf{v} \in \mathcal{T}_{\mathcal{V}}(\mathbf{s}_i)} P(\mathbf{V} = \mathbf{v} \mid \mathbf{S} = \mathbf{s}_i; \phi^{(n)})\, c_v(\mathbf{v})}_{\hat{c}_v(\mathcal{C}; \phi^{(n)})\text{: expected count of } v \text{ in the corpus}} \log \phi_v$$

- The E-step thus reduces to collecting these expected counts for all tokens in $\mathcal{V}$, which can be computed efficiently with the **forward-backward algorithm**

M-STEP

- Goal: find the parameters that maximize $Q(\phi; \phi^{(n)})$, subject to the token probabilities summing to 1
- Same as MLE for a categorical distribution, but with expected instead of observed counts:

$$\phi_v^{(n+1)} = \frac{\hat{c}_v(\mathcal{C}; \phi^{(n)})}{\sum_{v' \in \mathcal{V}} \hat{c}_{v'}(\mathcal{C}; \phi^{(n)})}$$

EXAMPLE (CONTINUED)

- Corpus $\mathcal{C} = \{\textit{hat}\}$, uniform initialization $\phi_v^{(0)} = 0.2$ for all $v \in \mathcal{V}$
- Segmentation probabilities and posteriors, where we use $P(\mathbf{S} = \textit{hat}; \boldsymbol{\phi}^{(0)}) = 0.088$:

$\mathbf{v}$	$P(\mathbf{V} = \mathbf{v}; \boldsymbol{\phi}^{(0)})$	$P(\mathbf{V} = \mathbf{v} \mid \mathbf{S} = \textit{hat}; \boldsymbol{\phi}^{(0)})$
$\langle \mathtt{h}, \mathtt{a}, \mathtt{t} \rangle$	$0.2^3 = 0.008$	$1/11 \approx 0.091$
$\langle \mathtt{ha}, \mathtt{t} \rangle$	$0.2^2 = 0.04$	$5/11 \approx 0.455$
$\langle \mathtt{h}, \mathtt{at} \rangle$	$0.2^2 = 0.04$	$5/11 \approx 0.455$

- For the expected counts, we weight each segmentation's token counts by its posterior, e.g.,

$$\hat{c}_{\mathtt{h}}(\mathcal{C}; \boldsymbol{\phi}^{(0)}) = 1 \cdot \tfrac{1}{11} + 1 \cdot \tfrac{5}{11} = \tfrac{6}{11} \approx 0.545$$

EXAMPLE (CONTINUED)

- Normalizing the expected counts gives the new parameters:

	h	a	t	ha	at
$\phi_v^{(0)}$	0.2	0.2	0.2	0.2	0.2
$\hat{c}_v(\mathcal{C}; \phi^{(0)})$	0.545	0.091	0.545	0.455	0.455
$\phi_v^{(1)}$	0.261	0.043	0.261	0.217	0.217

- a only appears in the low-posterior segmentation $\langle \mathtt{h}, \mathtt{a}, \mathtt{t} \rangle$, so its probability drops sharply
- h and t gain the most: each appears in two segmentations, including a high-posterior one

PRUNING

- EM alone does not change the vocabulary (it only re-estimates token probabilities over a fixed $\mathcal{V}$)
- Every few EM iterations, remove a fraction of tokens (e.g., 20%) and renormalize ϕ, until the target size is reached
- Remove the tokens whose removal lowers the expected complete data log-likelihood the least (ϕ_{-v}: ϕ without v, renormalized):

$$L(v) = Q(\phi^{(n+1)}; \phi^{(n)}) - Q(\phi_{-v}^{(n+1)}; \phi_{-v}^{(n)})$$

- Exact computation of $L(v)$ needs a forward-backward pass per token, so it is approximated: replace every use of v by the best segmentation of its string without v

EMPIRICAL COMPARISON OF BPE AND UNIGRAM

- Unigram results in segmentations that **align more closely with morphology** than BPE ▸ Hofmann et al. (2022)[3]
- This can lead to better performance ▸ Bostrom and Durrett (2020)[2]

Original:	furiously	**Original:**	tricycles	**Original:**	nanotechnology
BPE:	_fur iously	**BPE:**	_t ric y cles	**BPE:**	_n an ote chn ology
Uni. LM:	_fur ious ly	**Uni. LM:**	_tri cycle s	**Uni. LM:**	_nano technology

Original:	Completely preposterous suggestions
BPE:	_Comple t ely _prep ost erous _suggest ions
Unigram LM:	_Complete ly _pre post er ous _suggestion s

Original:	corrupted	**Original:**	1848 and 1852,
BPE:	_cor rupted	**BPE:**	_184 8 _and _185 2,
Unigram LM:	_corrupt ed	**Unigram LM:**	_1848 _and _1852 ,

Original	磁性は	様々に	分類がなされている。				
BPE	磁	性は	様々	に	分類	がなされている	。
Unigram LM	磁	性	は	様々	に	分類	がなされている	。
Gloss	magnetism	(top.)	various ways	in	classification	is done	.
Translation	Magnetism is classified in various ways.						

OUTLINE

Subword Tokenization Basics

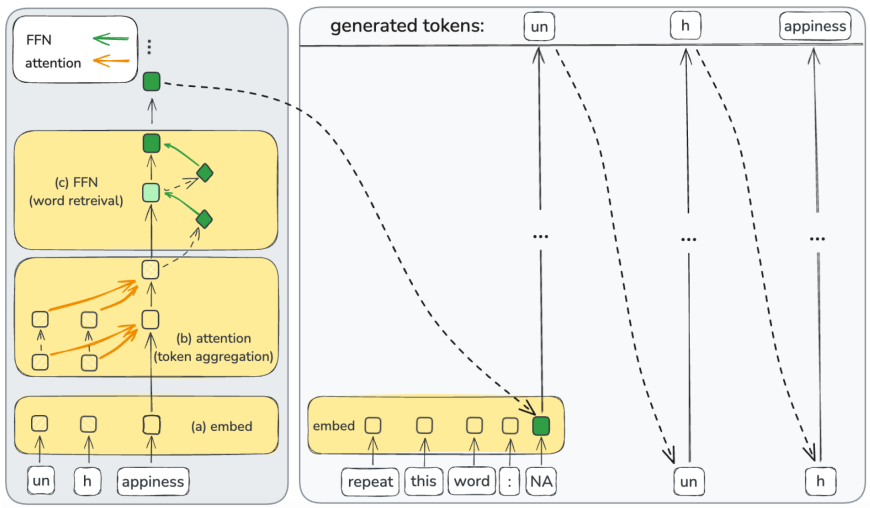
Overview of BPE

Overview of Unigram

Current Issues in Tokenization Research

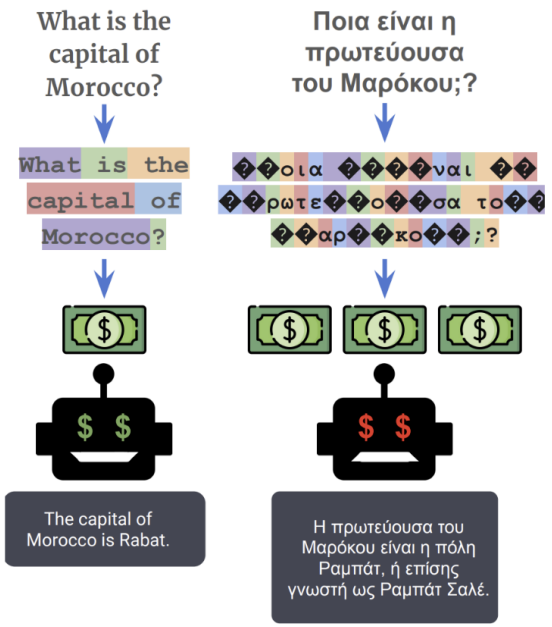
TOKENIZATION AND INTERPRETABILITY

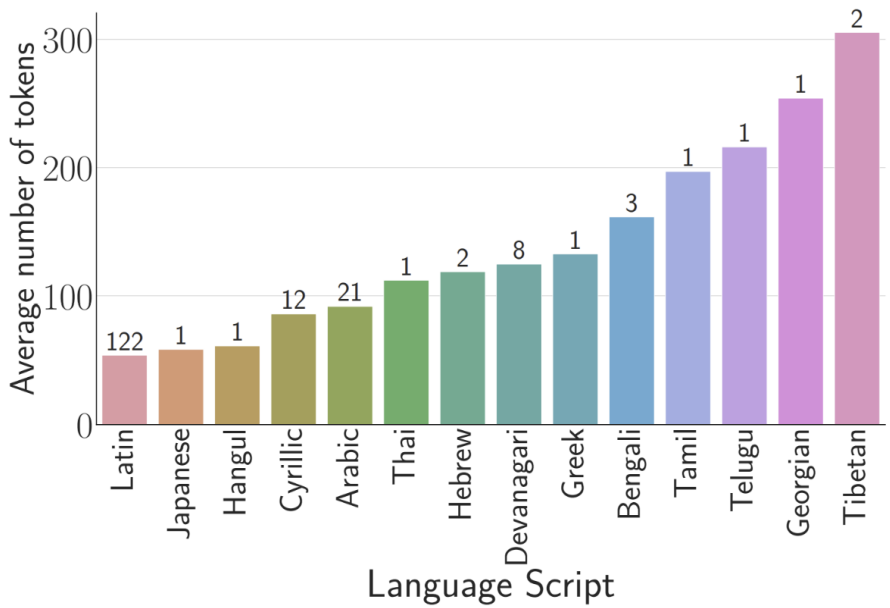
- Morphologically complex words (e.g., *unhappiness*) tend to be segmented into several tokens by BPE and Unigram, but LLMs still "understand" their meaning as a whole
- LLMs possess a **soft lexicon** encoded by the model weights that is much bigger than the **hard lexicon** defined by $\mathcal{V}$
- The mapping between the hard and the soft lexicon is still poorly understood ▸ Kaplan et al. (2025)[4]



MULTILINGUAL UNFAIRNESS

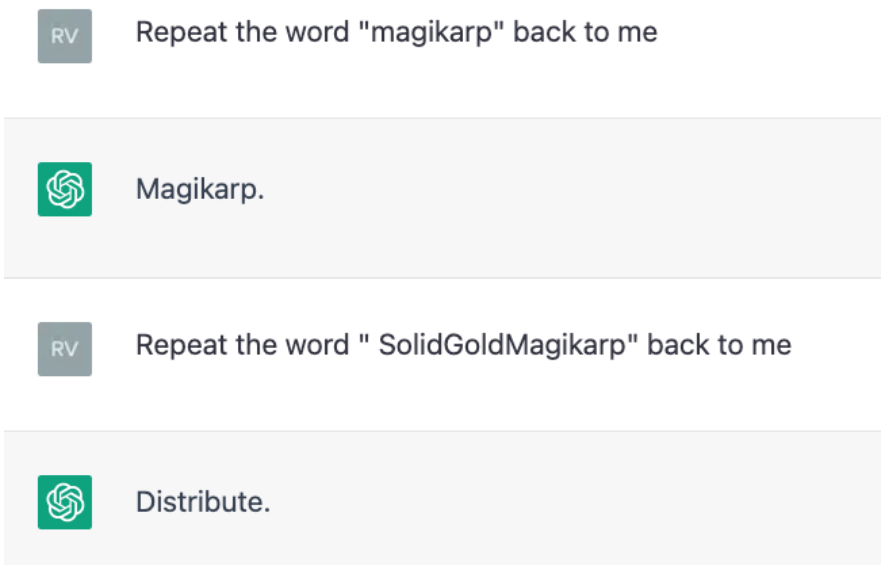
- Tokenizers trained mostly on English data split the *same* text into many more tokens in other languages, especially **low-resource languages** and **non-Latin scripts**
- Longer sequences mean **higher costs**, **slower inference**, and less text fitting into the context window, which can **hurt downstream performance** ▸ Ahia et al. (2023)[1]





GLITCH TOKENS

- The tokenizer training data can differ from the LLM training data
- This can lead to undertrained tokens with unwanted behavior, so-called **glitch tokens** ▸ Land and Bartolo (2024)[6]



Can we dispense with the tokenizer altogether?

Stay tuned for week 5!

REFERENCES

[1] Orevaoghene Ahia, Sachin Kumar, Hila Gonen, Jungo Kasai, David Mortensen, Noah Smith, and Yulia Tsvetkov. Do all languages cost the same? tokenization in the era of commercial language models. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 9904–9923, Singapore, December 2023. Association for Computational Linguistics.

[2] Kaj Bostrom and Greg Durrett. Byte pair encoding is suboptimal for language model pretraining. In Trevor Cohn, Yulan He, and Yang Liu, editors, *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4617–4624, Online, November 2020. Association for Computational Linguistics.

[3] Valentin Hofmann, Hinrich Schuetze, and Janet Pierrehumbert. An embarrassingly simple method to mitigate undesirable properties of pretrained language model tokenizers. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio, editors, *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 385–393, Dublin, Ireland, May 2022. Association for Computational Linguistics.

[4] Guy Kaplan, Matanel Oren, Yuval Reif, and Roy Schwartz. From tokens to words: On the inner lexicon of LLMs. In *The Thirteenth International Conference on Learning Representations*, 2025.

[5] Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates. In Iryna Gurevych and Yusuke Miyao, editors, *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 66–75, Melbourne, Australia, July 2018. Association for Computational Linguistics.

[6] Sander Land and Max Bartolo. Fishing for magikarp: Automatically detecting under-trained tokens in large language models. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 11631–11646, Miami, Florida, USA, November 2024. Association for Computational Linguistics.

[7] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In Katrin Erk and Noah A. Smith, editors, *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany, August 2016. Association for Computational Linguistics.